Web Technologies

.....AND SECURITY

Stones Dalitso Chindipha

Chapter 6

JavaScript: Client-Side Scripting

Objectives

1. What is JavaScript
2. JavaScript Design
3. Using JavaScript
4. JavaScript Syntax
5. JavaScript Objects
6. The DOM
7. JavaScript Events
8. Forms

The Document Object Model (DOM)

- When a web page is loaded, the browser creates a Document Object Model of the page.
- With the **HTML DOM**, JavaScript can access and change all the elements of an HTML document.
- JavaScript is almost always used to interact with the HTML document in which it is contained.
  - This is accomplished through a programming interface (API) called the Document Object Model.
- With the object model, JavaScript gets all the power it needs to create dynamic HTML:
  - **change all the HTML elements in the page, change all the HTML attributes in the page**
  - **Change all the CSS styles in the page, remove existing HTML elements and attributes**
  - **add new HTML elements and attributes, react to all existing HTML events in the page**
  - **create new HTML events in the page**

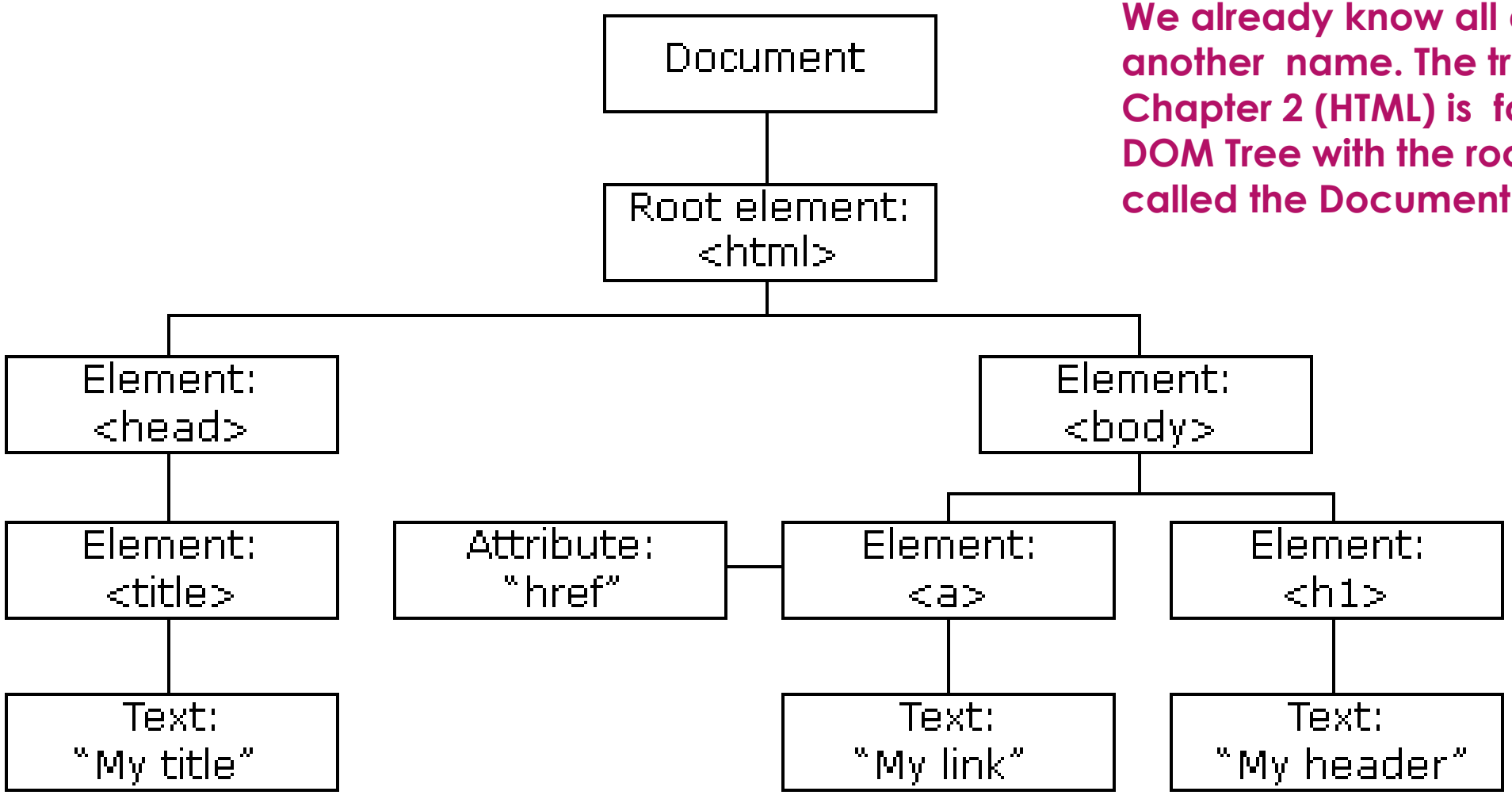
What is the DOM

- According to W3C (World Wide Web Consortium) standard:
  - ***The W3C Document Object Model (DOM) is a platform and language-neutral interface that allows programs and scripts to dynamically access and update the content, structure, and style of a document***
- The W3C DOM standard is separated into 3 different parts:
  - Core DOM - standard model for all document types
  - XML DOM - standard model for XML documents
  - **HTML DOM - standard model for HTML documents**

https://www.w3schools.com/js/js_htmldom.asp

The HTML DOM Tree of Objects

We already know all about the DOM, but by another name. The tree structure from Chapter 2 (HTML) is formally called the DOM Tree with the root, or topmost object called the Document Root.



```
<html>
<head lang="en">
   <meta charset="utf-8">
   <title>Share Your Travels</title>
</head>
<body>
   <h1>Share Your Travels</h1>
   <p>Photo of Conservatory Pond in
      <a href="http://www.centralpark.com/">Central Park</a>
   </p>
   <img src="images/central-park.jpg" alt="Central Park" />
   <h2>Reviews</h2>
   <div id="latestComment">
      <p>By Ricardo on <time>September 15, 2012</time></p>
      <p>Easy on the HDR buddy.</p>
   </div>
   <div>
      <p>By Susan on <time>October 1, 2012</time></p>
      <p>I love Central Park.</p>
   </div>
</body>
</html>
```

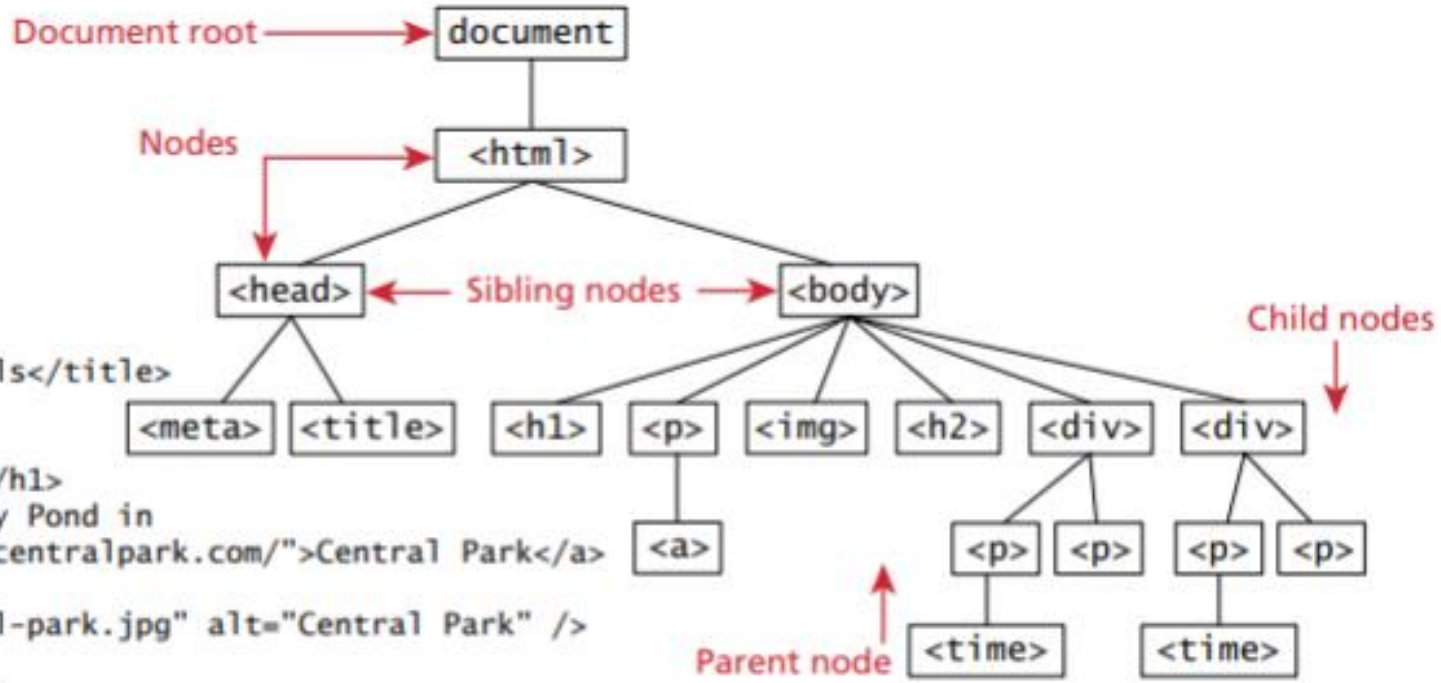


FIGURE 6.17 DOM tree

What is the HTML DOM?

- **HTML DOM** is a standard object model and programming interface for HTML.
- It defines: **The HTML elements as objects, The properties of all HTML elements, The methods to access all HTML elements, The events for all HTML elements.**
  - **i.e. The HTML DOM is a standard for how to get, change, add, or delete HTML elements.**
- In the DOM, each element within the HTML document is called a node.
- If the DOM is a tree, then each node is an individual branch.
- There are:
  - element nodes,
  - text nodes, and
  - attribute nodes
- All nodes in the DOM share a common set of properties and methods

```
<p>Photo of Conservatory Pond in
   <a href="http://www.centralpark.com/">Central Park</a>
</p>
```

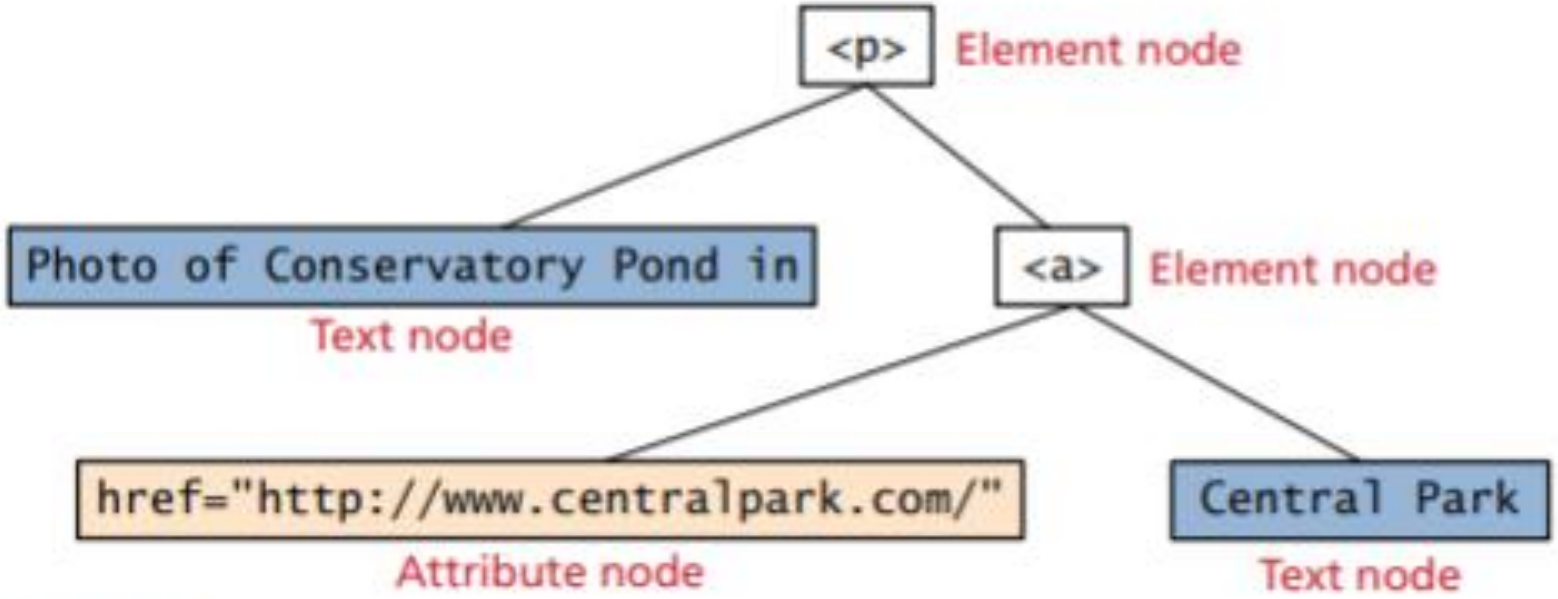


FIGURE 6.18 DOM nodes

Property	Description
`attributes`	Collection of node attributes
`childNodes`	A `NodeList` of child nodes for this node
`firstChild`	First child node of this node
`lastChild`	Last child of this node
`nextSibling`	Next sibling node for this node
`nodeName`	Name of the node
`nodeType`	Type of the node
`nodeValue`	Value of the node
`parentNode`	Parent node for this node
`previousSibling`	Previous sibling node for this node.

TABLE 6.3 Some Essential Node Object Properties

JavaScript - HTML DOM Methods

- HTML DOM methods are actions you can perform (on HTML Elements).
  - E.g. add or deleting an HTML element.
- HTML DOM properties are values (of HTML Elements) that you can set or change.
  - E.g. changing the content of an HTML element
- The **getElementById Method**: The most common way to access an HTML element is to use the **id** of the element.
- The **innerHTML Property**: The easiest way to get the content of an element is by using the **innerHTML** property.
  - The **innerHTML property** is useful for getting or replacing the content of HTML elements.
  - The **innerHTML property** can be used to get or change any HTML element, including **<html>** and **<body>.**

```html
<!DOCTYPE html>
<html>
<body>

<h2>My First Page</h2>

<p id="demo"></p>

<!-- This example changes the content (the innerHTML) of the <p> element with
id="demo".

In this example, getElementById is a method, while innerHTML is a property. -->

<script>
document.getElementById("demo").innerHTML = "Hello World!";
</script>

</body>
</html>
```

Method	Description
`createAttribute()`	Creates an attribute node
`createElement()`	Creates an element node
`createTextNode()`	Creates a text node
`getElementById(id)`	Returns the element node whose `id` attribute matches the passed `id` parameter
`getElementsByTagName(name)`	Returns a NodeList of elements whose tag name matches the passed name parameter

TABLE 6.4 Some Essential Document Object Methods

HTML DOM Document Object

- The **DOM document object** is the root JavaScript object representing the entire HTML document.
  - I.e. The **HTML DOM document object** is the owner of all other objects in your web page.
  - It contains some properties and methods are used extensively in web development and is globally accessible as document.
- The document object represents your web page.
  - If you want to access any element in an HTML page, you always start with accessing the document object.

Some examples of how you can use the document object to access and manipulate HTML.

Finding HTML Elements

Method	Description
document.getElementById(*id*)	Find an element by element id
document.getElementsByTagName(*name*)	Find elements by tag name
document.getElementsByClassName(*name*)	Find elements by class name

Changing HTML Elements

Property	Description
element.innerHTML = *new html content*	Change the inner HTML of an element
element.attribute = *new value*	Change the attribute value of an HTML element
element.style.*property* = *new style*	Change the style of an HTML element
Method	**Description**
element.setAttribute*(attribute, value)*	Change the attribute value of an HTML element

Adding and Deleting Elements

Method	Description
document.createElement(*element*)	Create an HTML element
document.removeChild(*element*)	Remove an HTML element
document.appendChild(*element*)	Add an HTML element
document.replaceChild(*new, old*)	Replace an HTML element
document.write(*text*)	Write into the HTML output stream

JavaScript HTML DOM Elements

- JavaScript manipulate HTML elements by finding the elements first.
- There are several ways to finding the elements: Finding HTML elements by:
  - Id, Tag name, class name, CSS selectors, HTML object collections
- **Finding HTML Element by Id:** Its easiest way to find an HTML element in the DOM
  - If the element is found, the method will return the element as an object (in element).
  - If the element is not found, element will contain null.
  - `const element = document.getElementById("intro");`

```html
<!DOCTYPE html>
<html>
<body>

<h2>My First Page</h2>

<p id="demo"></p>

<!-- This example changes the content (the innerHTML) of the <p> element with
id="demo".

In this example, getElementById is a method, while innerHTML is a property. -->

<script>
document.getElementById("demo").innerHTML = "Hello World!";
</script>

</body>
</html>
```

```html
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM</h2>

<!-- This example demonstrates the getElementsById method -->


<p id="intro">Finding HTML Elements by Id</p>

<p id="demo"></p>

<script>
const element = document.getElementById("intro");

document.getElementById("demo").innerHTML =
"The text from the intro paragraph is: " + element.innerHTML;

</script>

</body>
</html>
```

Finding HTML Elements by Tag Name

- Finding HTML Elements by Tag Name uses whatever tag you have supplied as your criteria
  - ```
    const element = document.getElementsByTagName("p");
    ```
  - ```
    const x = document.getElementById("main");
    const y = x.getElementsByTagName("p");
    ```

```html
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM</h2>

<p>Finding HTML Elements by Tag Name.</p>
<!-- This example demonstrates the <b>getElementsByTagName</b> method. -->

<p id="demo"></p>

<script>
const element = document.getElementsByTagName("p");

document.getElementById("demo").innerHTML = 'The text in first paragraph
(index 0) is: ' + element[0].innerHTML;

</script>

</body>
</html>
```

```html
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM</h2>

<!-- This example finds the element with id="main", and then finds all <p>
elements inside "main":-->
<div id="main">
<p>Finding HTML Elements by Tag Name</p>
<p>This example demonstrates the <b>getElementsByTagName</b> method.</p>
</div>

<p id="demo"></p>

<script>
const x = document.getElementById("main");
const y = x.getElementsByTagName("p");

document.getElementById("demo").innerHTML =
'The first paragraph (index 0) inside "main" is: ' + y[0].innerHTML;

</script>

</body>
</html>
```

Finding HTML Elements by Class Name

- If you want to find all HTML elements with the same class name, use **getElementsByClassName()**.
  - ```
    const x = document.getElementsByClassName("intro");
    ```
    ```

```html
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM</h2>

<!-- This example returns a list of all elements with class="intro"-->

<p>Finding HTML Elements by Class Name.</p>
<p class="intro">Hello World!</p>
<p class="intro">This example demonstrates the <b>getElementsByClassName</b>
method.</p>

<p id="demo"></p>

<script>
const x = document.getElementsByClassName("intro");
document.getElementById("demo").innerHTML =
'The first paragraph (index 0) with class="intro" is: ' + x[0].innerHTML;
</script>

</body>
</html>
```

JavaScript HTML DOM - Changing HTML

- The HTML DOM allows JavaScript to change the content of HTML elements by using the innerHTML property.
  - It's the easiest way to modify the content of an HTML element
    - This includes headers, changing the Value of an Attribute like background colours etc
  - To change the content of an HTML element, use this syntax:
    - `document.getElementById(id).innerHTML = new HTML`
  - To change the value of an HTML attribute, use this syntax:
    - `document.getElementById(id).attribute = new value`

```html
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript can Change HTML</h2>

<!-- This example changes the content of a <p> element:-->

<p id="p1">Hello World!</p>

<script>
document.getElementById("p1").innerHTML = "New text!";
</script>

<p>The paragraph above was changed by a script.</p>

</body>
</html>
```

```html
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML DOM</h2>

<!-- This example changes the value of the src attribute of an <img> element:-->
<img id="image" src="smiley.gif" width="160" height="120">

<script>
document.getElementById("image").src = "landscape.jpg";
</script>

<p>The original image was smiley.gif, but the script changed it to
landscape.jpg</p>

</body>
</html>
```

JavaScript can create dynamic HTML content:

```html
<!DOCTYPE html>
<html>
<body>

<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML = "Date : " + Date();
</script>

</body>
</html>
```

Date : Thu May 12 2022 04:10:38 GMT+0200 (GMT+02:00)

In JavaScript, document.write() can be used to write directly to the HTML output stream:

```
<!DOCTYPE html>
<html>
<body>

<p>Bla, bla, bla</p>

<!--Never use document.write() after the document is loaded
It will overwrite the document. -->

<script>
document.write(Date());
</script>

<p>Bla, bla, bla</p>

</body>
</html>
```

JavaScript HTML DOM Events

- HTML DOM allows JavaScript to react to HTML events:
- A **JavaScript event** is an action that can be detected by JavaScript.
  - An event is triggered and then it can be caught by JavaScript functions, which then do something in response.
- A JavaScript can be executed when an event occurs e.g when a user clicks on an HTML element.
- To execute code when a user clicks on an element, add JavaScript code to an HTML event attribute:
  - `onclick=JavaScript`

Examples of HTML events:

- There are many events that can trigger JS to do something. For example when:
  - A user clicks the mouse
  - A web page has loaded
  - An image has been loaded
  - The mouse moves over an element
  - An input field is changed
  - An HTML form is submitted
  - A user strokes a key

Event	Description
`onclick`	The mouse was clicked on an element
`ondblclick`	The mouse was double clicked on an element
`onmousedown`	The mouse was pressed down over an element
`onmouseup`	The mouse was released over an element
`onmouseover`	The mouse was moved (not clicked) over an element
`onmouseout`	The mouse was moved off of an element
`onmousemove`	The mouse was moved while over an element

TABLE 6.7 Mouse Events in JavaScript

Event	Description
onkeydown	The user is pressing a key (this happens first)
onkeypress	The user presses a key (this happens after onkeydown)
onkeyup	The user releases a key that was down (this happens last)

TABLE 6.8 Keyboard Events in JavaScript

```html
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML Events</h2>
<h2 onclick="this.innerHTML='Ooops!'">Click on this text!</h2>

</body>
</html>
```

```html
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML Events</h2>
<h2 onclick="changeText(this)">Click on this text!</h2>

<!--In this example, a function is called from the event handler: -->
<script>
function changeText(id) {
  id.innerHTML = "Ooops!";
}
</script>

</body>
</html>
```

HTML Event Attributes

- There are several ways to assign events to allow JS to react to triggers
- To assign events to HTML elements you can use **event attributes**.
  - To Assign an **onclick event** to a button element:
    - ```
      <button onclick="displayDate()">Try it</button>
      ```
- The **HTML DOM** allows you to assign events to HTML elements using JavaScript:
  - Assign an onclick event to a button element:
    - ```
      <script>
      document.getElementById("myBtn").onclick = displayDate;
      </script>
      ```

```html
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML Events</h2>
<p>Click the button to display the date.</p>

<!--In the example above, a function named displayDate will be executed when the
button is clicked. -->

<button onclick="displayDate()">The time is?</button>

<script>
function displayDate() {
  document.getElementById("demo").innerHTML = Date();
}
</script>

<p id="demo"></p>

</body>
</html>
```

```html
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML Events</h2>
<p>Click "Try it" to execute the displayDate() function.</p>

<!--In this example, a function named displayDate is assigned to an HTML element
with the id="myBtn". -->

<button id="myBtn">Try it</button>

<p id="demo"></p>

<script>
document.getElementById("myBtn").onclick = displayDate;

function displayDate() {
  document.getElementById("demo").innerHTML = Date();
}
</script>

</body>
</html>
```

The, onchange, onload and onunload Events

- The **onload** and **onunload events** are triggered when the user enters or leaves the page.
- The **onload event** can be used to check the visitor's browser type and browser version, and load the proper version of the web page based on the information.
- The **onload** and **onunload** events can be used to deal with cookies.
  - `<body onload="checkCookies()“`
- The **onchange event** is often used in combination with validation of input fields.
  - The **upperCase()** function will be called when a user changes the content of an input field.
  - `<input type="text" id="fname" onchange="upperCase()">`

```html
<!DOCTYPE html>
<html>
<body onload="checkCookies()">

<h2>JavaScript HTML Events</h2>

<p id="demo"></p>

<script>
function checkCookies() {
  var text = "";
  if (navigator.cookieEnabled == true) {
    text = "Cookies are enabled.";
  } else {
    text = "Cookies are not enabled.";
  }
  document.getElementById("demo").innerHTML = text;
}
</script>

</body>
</html>
```

```html
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML Events</h2>

Enter your name: <input type="text" id="fname" onchange="upperCase()">

<!-- When you leave the input field, a function is triggered which transforms the input text to upper case -->

<script>
function upperCase() {
  const x = document.getElementById("fname");
  x.value = x.value.toUpperCase();
}
</script>

</body>
</html>
```

JavaScript HTML DOM EventListener

- In the original JavaScript world, events could be specified right in the HTML markup with hooks to the JavaScript code (and still can).
  - ** Click for pop-up **
- As more powerful frameworks were developed, and website design and best practices were refined, this original mechanism was supplanted by the **listener approach**

```
var greetingBox = document.getElementById('example1');
greetingBox.onclick = alert('Good Morning');
```

LISTING 6.10 The "old" style of registering a listener.

```
var greetingBox = document.getElementById('example1');
greetingBox.addEventListener('click', alert('Good Morning'));
greetingBox.addEventListener('mouseOut', alert('Goodbye'));

// IE 8
greetingBox.attachEvent('click', alert('Good Morning'));
```

LISTING 6.11 The "new" DOM2 approach to registering listeners.

The addEventListener() method

- The **addEventListener()** method attaches an event handler to the specified element.
- The **addEventListener()** method attaches an event handler to an element without overwriting existing event handlers.
- You can add many event handlers to one element.
- You can add many event handlers of the same type to one element, i.e two "click" events.
- You can add event listeners to any DOM object not only HTML elements. i.e the window object.
- **The addEventListener()** method makes it easier to control how the event reacts to bubbling.
- You can easily remove an event listener by using the **removeEventListener()** method.
- Syntax: `element.addEventListener(event, function, useCapture);`
  - `element.addEventListener("click", function(){ alert("Hello World!"); });`

https://www.w3schools.com/js/js_htmldom_eventlistener.asp

```html
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript addEventListener()</h2>

<!-- This example uses the addEventListener() method to attach a click event to a
button. -->

<button id="myBtn">Try it</button>

<p id="demo"></p>

<script>
document.getElementById("myBtn").addEventListener("click", displayDate);

function displayDate() {
  document.getElementById("demo").innerHTML = Date();
}
</script>

</body>
</html>
```

```html
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript addEventListener()</h2>

<!-- This example uses the addEventListener() method to execute a function when a
user clicks on a button. -->

<button id="myBtn">Try it</button>

<script>
document.getElementById("myBtn").addEventListener("click", myFunction);

function myFunction() {
  alert ("Hello World!");
}
</script>

</body>
</html>
```

```html
<body>

<h2>JavaScript addEventListener()</h2>

<!--This example uses the addEventListener() method to add two click events to
the same button. The addEventListener() method allows you to add many events to
the same element, without overwriting existing events -->

<button id="myBtn">Try it</button>

<script>
var x = document.getElementById("myBtn");
x.addEventListener("click", myFunction);
x.addEventListener("click", someOtherFunction);

function myFunction() {
  alert ("Hello World!");
}

function someOtherFunction() {
  alert ("This function was also executed!");
}
</script>

</body>
</html>
```

```html
<!-- You can add events of different types to the same element:This example uses
the addEventListener() method to add many events on the same button.-->

<button id="myBtn">Try it</button>

<p id="demo"></p>

<script>
var x = document.getElementById("myBtn");
x.addEventListener("mouseover", myFunction);
x.addEventListener("click", mySecondFunction);
x.addEventListener("mouseout", myThirdFunction);

function myFunction() {
  document.getElementById("demo").innerHTML += "Moused over!<br>";
}

function mySecondFunction() {
  document.getElementById("demo").innerHTML += "Clicked!<br>";
}

function myThirdFunction() {
  document.getElementById("demo").innerHTML += "Moused out!<br>";
}
</script>
```

JavaScript Event Handlers

- Event handlers can be used to handle and verify user input, user actions, and browser actions:
  - Things that should be done every time a page loads
  - Things that should be done when the page is closed
  - Action that should be performed when a user clicks a button
  - Content that should be verified when a user inputs data
  - And more ...

JavaScript Form Validation

- Writing code to pre-validate forms on the client side will reduce the number of incorrect submissions, thereby reducing server load.
- There are a number of common validation activities including email validation, number validation, and data validation.
- HTML form validation can be done by JavaScript.
- If a form field (fname) is empty, this function alerts a message, and returns false, to prevent the form from being submitted:

```
function validateForm() {
  let x = document.forms["myForm"]["fname"].value;
  if (x == "") {
    alert("Name must be filled out");
    return false;
  }
}
```

```html
<h2>JavaScript Validation</h2>

<p>Please input a number between 1 and 10:</p>

<input id="numb">

<button type="button" onclick="myFunction()">Submit</button>

<p id="demo"></p>
```

```javascript
<script>
function myFunction() {
  // Get the value of the input field with id="numb"
  let x = document.getElementById("numb").value;
  // If x is Not a Number or less than one or greater than 10
  let text;
  if (isNaN(x) || x < 1 || x > 10) {
    text = "Input not valid";
  } else {
    text = "Input OK";
  }
  document.getElementById("demo").innerHTML = text;
}
</script>
```

```
document.getElementById("loginForm").onsubmit = function(e){
   var fieldValue=document.getElementByID("username").value;
   if(fieldValue==null || fieldValue== ""){
      // the field was empty. Stop form submission
      e.preventDefault();
      // Now tell the user something went wrong
      alert("you must enter a username");
   }
}
```

LISTING 6.18 A simple validation script to check for empty fields

```
var inputField=document.getElementByID("license");
if (inputField.type=="checkbox"){
 if (inputField.checked)
 //Now we know the box is checked, otherwise it isn't
}
```

Data Validation

- **Data validation** is the process of ensuring that user input is clean, correct, and useful.
- Typical validation tasks are:
  - has the user filled in all required fields?
  - has the user entered a valid date?
  - has the user entered text in a numeric field?
- Most often, the purpose of data validation is to ensure correct user input.
- Validation can be defined by many different methods, and deployed in many different ways.
- **Server side validation** is performed by a web server, after input has been sent to the server.
- **Client side validation** is performed by a web browser, before input is sent to a web server.

Reading Task

- JavaScript HTML DOM Elements (Nodes), JavaScript HTML DOM Collections, JavaScript HTML DOM Node Lists, The Difference Between an HTMLCollection and a NodeList, JavaScript Window Navigator, JavaScript Cookies
  - Please note that the reading Task above is NOT for today's Quiz but can come in handy for your practical and can examined or tested, just not today
    - Content found in your book chapter 6 & w3schools